

The C++ Model of Concurrency

A CS 787 Lecture

Rushikesh Joshi
IIT Bombay

Tasks and Threads

Task = computation that is potentially concurrent with other tasks

Thread = system level representation of a task

A concurrent task can be launched by `std::thread` with task as its argument

Task is represented as (i) a function or (ii) a function object

Class thread

```
#include <iostream>

#include <thread>

using namespace std;

void f() { while (1) cout << "hello\n";}

void g() { while (1) cout << "world\n";}

i
```

```
int main (int argc, char *argv[]) {

    thread t1 {f};

    thread t2 {g};

    t1.join();

    // due to inf loop, control will not reach here

    t2.join();

    // due to inf loop, control will not reach here

}

// g++-8 threads.cpp -lpthread
```

Sleep timers

```
void f() {  
    while (1) {  
        cout << "hello\n";  
        this_thread::sleep_for (chrono::milliseconds(300));  
    }  
}  
  
void g() {  
    while (1) {  
        cout << "world\n";  
        this_thread::sleep_for (chrono::milliseconds(500));  
    }  
}
```

This_thread namespace

`thread::id get_id()` returns this thread's id

`void yield()`

yields to other threads, implementation can reschedule

`sleep_until` sleep till a point in time

`sleep_for` sleep for given milliseconds

yield

```
void f() {  
  
    int count =0;  
  
    while (1) {  
        cout << count << "hello  *****\n";  
        count++;  
        if (!(count%4)) this_thread::yield();  
        this_thread::sleep_for  
(chrono::milliseconds(100));  
    }  
}
```

```
void g() {  
    while (1) {  
        cout << "world\n";  
        this_thread::sleep_for  
(chrono::milliseconds(300));  
    }  
}
```

Threads from Class members

```
int main (int argc, char *argv[]) {  
  
    A *a1 = new A(1000);  
  
    B *b1 = new B();  
  
    thread t1 {& A::f, a1};  
  
    thread t2 {& B::g, b1};  
  
  
    t1.join(); // due to inf loop, control will not reach  
    here  
  
    t2.join(); // due to inf loop, control will not reach  
    here  
  
}
```

```
// g++-8 threads.cpp -lpthread
```

```
class A {  
  
    int val;  
  
    public:  
  
    A(int x) {val = x;}  
  
    void f() { while (1) cout << (val++)%10  
    << endl;}  
  
};  
  
class B {  
  
    public:  
  
    void g() { while (1) cout << "world\n";}  
  
};
```

```
#include <iostream>
```

```
#include <thread>
```

```
#include <mutex>
```

```
using namespace std;
```

```
mutex lck;
```

```
void f() {
```

```
    while (1) {
```

```
        lck.lock();
```

```
        cout << "hello\n";    cout << "hello\n";    cout << "hello\n";
```

```
        lck.unlock();
```

```
        this_thread::sleep_for (chrono::milliseconds(300));
```

```
    }
```

```
} // similar code for g
```

lockable MUTEX

- Locking a mutex prevents

another thread from locking it until

it gets unlocked.

- Non-deterministic

- try_lock() locks mutex only if lockable,

else returns false